Checkpoint resume mechanics

How `app/main.py`'s checkpointer persists a paused `interrupt()` across two genuinely separate HTTP requests — and why that's the whole reason it's `AsyncPostgresSaver` and not `SqliteSaver`.

The problem this replaces

The prior checkpointer, `SqliteSaver`, wrote to a local file (`checkpoints.db`) on the backend's disk. On Render's free tier, that disk is **ephemeral** — every spin-down/spin-up cycle after idle time hands the app a fresh container, and local disk contents don't survive that recycle.

Concretely: a run pauses at `await_approval` waiting on a human; the free instance idles out before anyone clicks Approve; the next request boots a new container with an empty `checkpoints.db`. The paused thread's history is simply gone — `Command(resume=...)` has nothing to resume.

A checkpointer backed by a real database server isn't on the app host's disk at all. The container can be destroyed and rebuilt entirely — as long as `DATABASE_URL` still points somewhere real, every paused thread survives. That's the entire justification for the swap; nothing about `interrupt()`'s own logic changed.

The contract a checkpointer implements

LangGraph's runtime is written against one abstract interface — four async methods are all it needs, and it has no idea Postgres is underneath:

Method	Purpose
`aget_tuple()`	Fetch the latest checkpoint for a `thread_id` (or a specific one, given a `checkpoint_id`).
`aput()`	Persist a fully-merged state snapshot as a new checkpoint, linked to its parent.
`aput_writes()`	Stage one task's output before the whole superstep is merged — so a crash mid-superstep doesn't lose completed work.
`alist()`	Enumerate checkpoints for a thread — history/time-travel; not exercised by this app today.

The mechanism

